Introduction to Computer Networks

Recitation 01: Socket programming

Gabriel Domingues
gm@mail.tau.ac.il

Administration

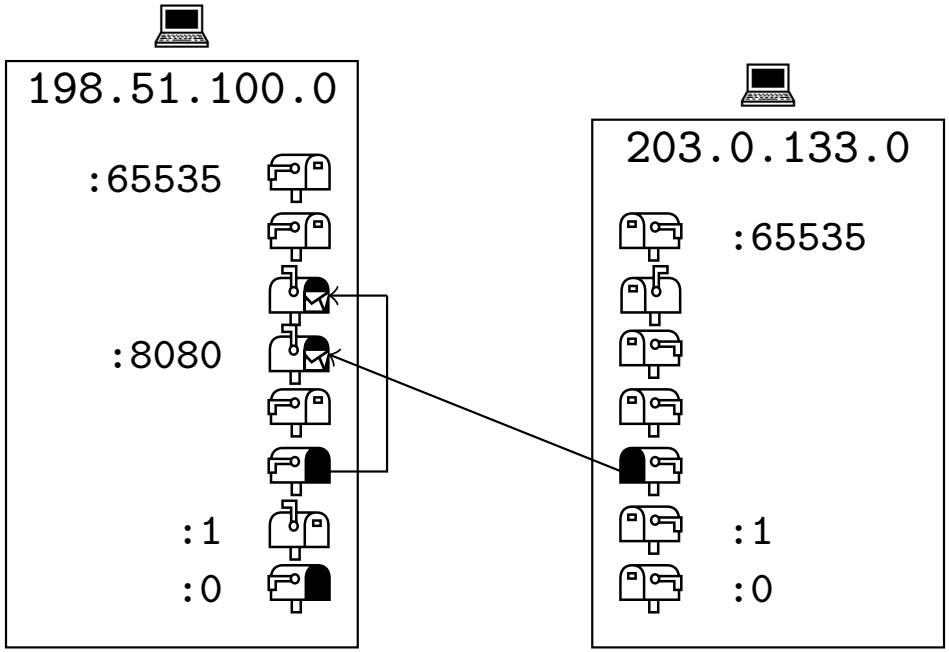
- Contact me
  - Moodle Q&A forum for general questions on the material and homework
  - Email for specific questions (late submissions, grading)
  - I always respond in English, but you can ask and email in Hebrew
- Teaching philosophy
  - This is engineering, we build things
  - Understanding the history helps
  - The manual is your friend
- The course is in C
  - C++ is acceptable for homework (with caveats)
  - My code demos are using the Julia programming language
- Unix/POSIX bias
  - If you are running GNU/Linux or a BSD (like MacOS), it is probably fine
  - If you are running Windows, try to install Windows Subsystem for Linux (WSL)
  - Otherwise, the computer lab has VirtualBox (instructions on Moodle)
  - Homework will be checked using Ubuntu

Today's recitation: Socket programming

- Preliminary concepts
  - Network address and ports
  - Sockets as file descriptors
- Berkeley sockets
- POSIX
  - Socket API
  - Multiplexing

Network addresses and ports

- Each computer has an IP address
  - Loopback/Localhost: 127.0.0.1
- Each computer has 2^{16} ports
- Some ports are reserved for special uses
  - FTP: 20,21; SSH: 22; HTTP: 80; etc.



Sockets are special files

Sockets

A socket is a file descriptor for the communication between addresses/ports.

files	`open`	`write`	`read`	`close`
sockets	`socket`	`send`	`recv`	

Creating a new socket

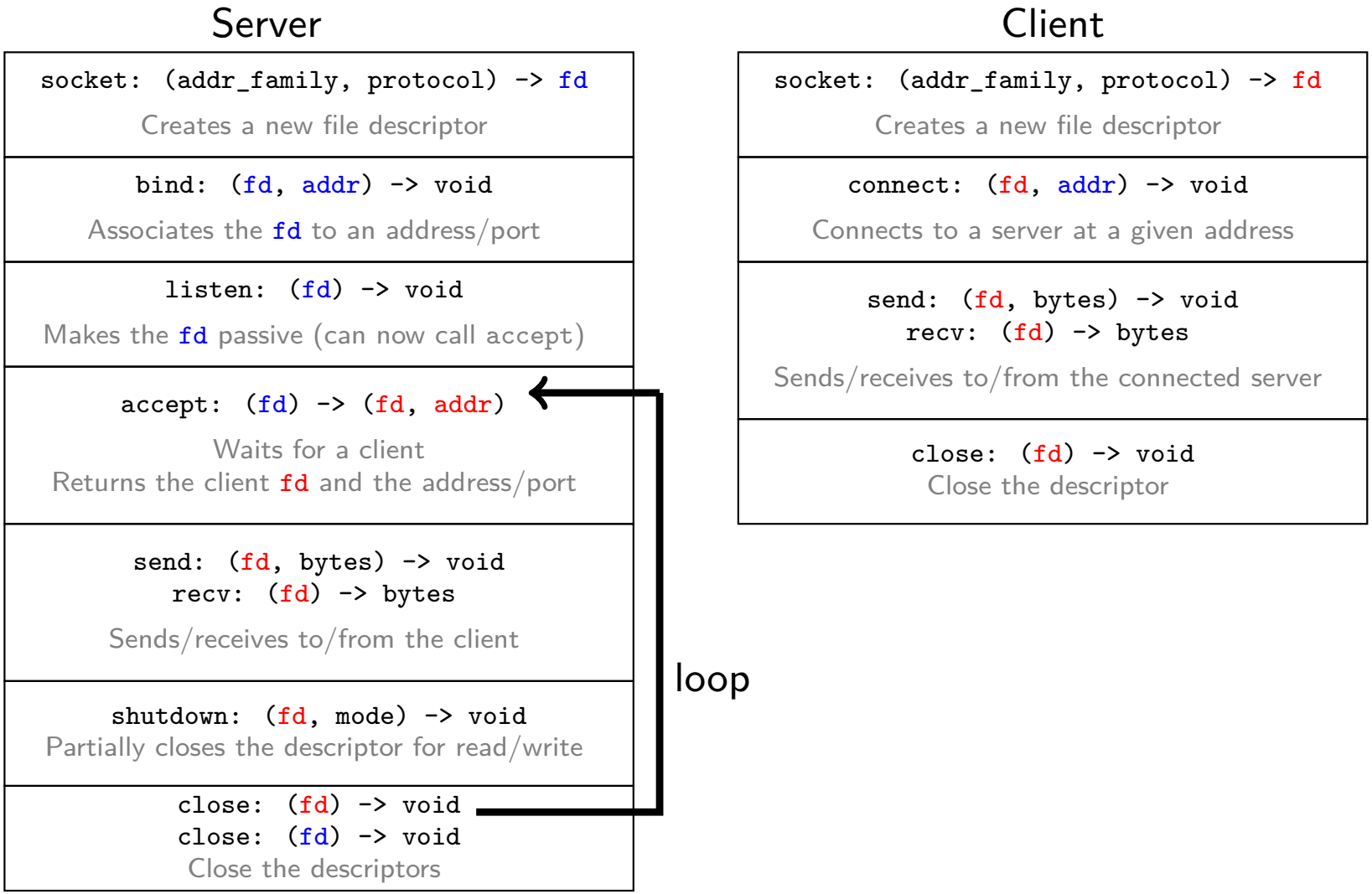
- `socket : (addr_family, protocol) -> fd`
- Upon creation, the socket is not associated to any address or port.

Relevant protocols

- Address family `AF_INET`
  - For example: `127.0.0.1` (loopback)
- Transmission Control Protocol (TCP)
  - Ordered and reliable, but slower
  - Web server, file transfer, email
  - Socket type `SOCK_STREAM`
- User Datagram Protocol (UDP)
  - Unordered and unreliable, but faster
  - Streaming audio and video
  - Socket type `SOCK_DGRAM`

Demo time!

Berkeley sockets (1983)



[1] Blue: Server `fd`; Red: Client `fd`.

POSIX sockets (1988)

Syscalls

- `int socket(int domain, int type, int protocol);`
- `int bind(int fd, const struct sockaddr *addr, socklen_t addrlen);`
- `int listen(int fd, int backlog);`
- `int accept(int fd, struct sockaddr *addr, socklen_t *addrlen);`
- `int connect(int fd, const struct sockaddr *addr, socklen_t addrlen);`
- `ssize_t send(int fd, const char *buff, size_t bufflen, int flags);`
- `ssize_t recv(int fd, char *buff, size_t bufflen, int flags);`
- `int shutdown(int fd, int how);`
- `int close(int fd);`

Address info

- `struct sockaddr_in { uint16_t sin_family; uint16_t sin_port; uint32_t sin_addr; uint8_t sin_zero[8]; /* padding */ };`
- Cast `(struct sockaddr *) &addr`, the syscall selects the layout based on the family.

[1] Green: Input parameter (read-only); Orange: Output parameter (write-only).

Tips and tricks

- The manual is your friend https://www.man7.org/linux/man-pages.
- Endianess: `htons/htonl` and `ntohs/ntohl`.

```
struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(8080),
        .sin_addr.s_addr = htonl(INADDR_LOOPBACK) };
uint16_t port = ntohs(addr.sin_port);
```

- Always use: `void perror(const char *msg);`
- `ENOENT` is "Error no entity" – the file descriptor does not exist.
- `EADDRINUSE` means that the port in use. Usually you did not close the previously bound fd.
  - Call `setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, opt, optlen)` when creating a socket.

Sample code

```c
int server(int port, const char *msg, size_t msglen) {
    int err;
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sfd == -1) { perror("socket()"); return 1; }
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(port),
        .sin_addr.s_addr = INADDR_ANY };
    err = bind(sfd, (const struct sockaddr *) &addr, sizeof addr);
    if (err == -1) { perror("bind()"); close(sfd); return -1; }
    err = listen(sfd, 5);
    if (err == -1) { perror("listen()"); close(sfd); return -1; }
    for (;;) { /* handle */
        struct sockaddr_in caddr;
        socklen_t clen = sizeof caddr;
        int cfd = accept(sfd, (struct sockaddr *) &caddr, &clen);
        if (cfd == -1) { perror("accept()"); break; }
        send(cfd, msg, msglen, 0);
        shutdown(cfd, SHUT_WR);
        close(cfd);
    }
    close(sfd);
    return 0;
}
```

Sample code

```c
int client(int port, int addr) {
    int err;
    int cfd = socket(AF_INET, SOCK_STREAM, 0);
    if (cfd == -1) { perror("socket()"); return 1; }
    struct sockaddr_in caddr = {
        .sin_family = AF_INET,
        .sin_port = htons(port),
        .sin_addr.s_addr = htonl(addr) };
    err = connect(cfd, (const struct sockaddr *) &caddr, sizeof caddr);
    if (err == -1) { perror("connect()"); close(cfd); return -1; }
    char buff[2048] = {0};
    recv(cfd, buff, sizeof buff, 0);
    printf("Message:\n%.*s\n", (int) sizeof buff, buff);
    close(cfd);
    return 0;
}
```

UDP

Syscalls

- `ssize_t sendto(int fd, const char *buff, size_t bufflen, int flags, struct sockaddr *addr, socklen_t addrlen);`
  - Conceptually: `connect` + `send`
- `ssize_t recvfrom(int fd, char *buff, size_t bufflen, int flags, struct sockaddr *addr, socklen_t *addrlen);`
  - Conceptually: `accept` + `recv` + clean-up

Select

- When listening to many sockets, we need a way to wait until one or more are available to read/write.
- `fd_set` is a structure that stores a set of file descriptors.
  - `int FD_ZERO(fd_set *set);`
  - `int FD_ISSET(int fd, fd_set *set);`
  - `int FD_SET(int fd, fd_set *set);`
  - `int FD_CLR(int fd, fd_set *set);`
- `int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);`

[1] Green: Input parameter (read-only); Orange: Output parameter (write-only); Purple: Input and output parameter (read-write).

Questions?